

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа 2

Выполнил:

Каменев Матвей

Группа J3214

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Необходимо привязать то, что было сделано в ЛР1, к внешнему API средствами `fetch/axios/xhr`, реализовать моковое API средствами JSON-сервера и подключить к нему авторизацию.

В рамках работы подключено внешнее открытое API курсов валют `open.er-api.com` для конвертации цен заказов разметки, а также моковый бэкенд на `json-server` с авторизацией через `json-server-auth` для регистрации, входа и работы с принятыми заказами.

Ход работы

1. Настройка мокового сервера

В проект добавлен `package.json` с зависимостями `json-server` (0.17) и `json-server-auth` (2.1). Сервер запускается командой `npm start`, которая выполняет:

```
json-server db.json -m ./node_modules/json-server-auth
```

Сервер работает на `http://localhost:3000`, фронтенд раздаётся локальным статическим сервером (`npx http-server .`).

Файл `db.json` содержит две коллекции:

- `users`: пользователи; при регистрации `json-server-auth` сам хэширует пароль алгоритмом `bcrypt`;
- `annotations`: заказы, принятые пользователем в работу: `userId`, название, тип, описание, обложка, цена в рублях (`priceRub`), суффикс цены (`«/ 100 др»`) и ссылка на страницу разметки.

2. Регистрация и вход (`js/auth-modals.js`)

Запросы отправляются методом `fetch` на эндпоинты `POST /login` и `POST /register` с заголовком `Content-Type: application/json`.

При успешном ответе сервер возвращает `accessToken` и объект `user`, которые сохраняются в `localStorage`, после чего пользователь перенаправляется в личный кабинет. Ошибки выводятся в блоке `alert-`

danger внутри модального окна, при этом различаются отказ сервера («Неверный email или пароль», «Ошибка регистрации. Email уже занят?») и недоступность сервера («Ошибка соединения с сервером»).

3. Навигационная панель

Функция `updateNavbar()` вызывается по событию `DOMContentLoaded`. Если в `localStorage` есть токен, кнопки «Вход» и «Регистрация» скрываются, а вместо них выводятся имя пользователя и кнопка «Выйти», которая удаляет токен и данные пользователя и возвращает на главную страницу.

4. Работа с закрытым API (js/api.js)

Общая логика запросов к защищённым ресурсам вынесена в отдельный модуль:

`getCurrentUser()` и `logout()` чтение и очистка сессии в `localStorage`;
`authFetch(path, options)` обёртка над `fetch`, которая подставляет базовый адрес API и заголовок `Authorization: Bearer <токен>`; при ответе 401 (истёкший токен) сессия сбрасывается и выполняется переход на главную;
`getMyAnnotations(query)` получение принятых заказов текущего пользователя.

5. Принятие заказа (js/accept-order.js)

Кнопки «Принять» на карточках главной страницы теперь обращаются к API. Если пользователь не авторизован, открывается модальное окно входа (`bootstrap.Modal.getOrCreateInstance(...).show()`). Иначе скрипт проверяет, не принят ли уже этот заказ (запрос с фильтром по `link`), и при необходимости создаёт запись запросом `POST /600/annotations`, собирая данные из карточки. После этого выполняется переход на страницу

разметки. При недоступности сервера пользователь видит сообщение об ошибке.

6. Личный кабинет (profile.html, js/profile.js)

На странице добавлена проверка авторизации: без токена пользователь перенаправляется на главную. Имя в шапке кабинета подставляется из данных пользователя.

Вкладка «Мои аннотации», которая в ЛР1 была статичной, теперь заполняется данными с сервера. Функция `loadAnnotations()` показывает состояние загрузки, сообщение об ошибке при недоступном сервере или пустое состояние со ссылкой на выбор заказа. Карточки создаются функцией `renderCard()`: разметка берётся из шаблона, а данные подставляются через `textContent`, что исключает внедрение HTML из ответа сервера. Счётчик аннотированных проектов во вкладке «Статистика» также обновляется по данным API.

7. Внешнее API курсов валют (js/currency.js)

Для конвертации цен используется открытое API `https://open.er-api.com/v6/latest/RUB`, возвращающее курсы относительно рубля. На главной странице рядом с фильтрами добавлен переключатель валют: группа радиокнопок Bootstrap (₽, \$, €).

Все элементы с ценами получили атрибуты `data-price-rub` (цена в рублях) и `data-price-suffix`. Функция `updatePrices()` проходит по ним и форматирует цену функцией `formatPrice()`: рубли через `toLocaleString('ru-RU')`, доллары и евро умножением на курс с двумя знаками после запятой. Кнопки \$ и € заблокированы, пока курсы не загружены. Выбранная валюта сохраняется в `localStorage` (с обработкой исключений на случай недоступного хранилища) и применяется на всех страницах: к ценам заказов, балансу и статистике в кабинете, бюджету и выплатам на странице проекта.

Вывод

В ходе лабораторной работы сайт, свёрстанный в LP1, был подключён к API. С помощью json-server и json-server-auth реализован моковый бэкенд с регистрацией, входом по JWT-токену и защищёнными маршрутами, доступными только владельцу данных. На клиенте реализованы хранение сессии, изменение интерфейса в зависимости от авторизации, принятие заказов и динамическая загрузка списка аннотаций в личном кабинете с обработкой состояний загрузки, ошибки и пустого списка.

Дополнительно подключено внешнее открытое API курсов валют, благодаря которому все цены на сайте можно отображать в рублях, долларах или евро. Все запросы выполнены средствами fetch с использованием async/await.